



Binary Search

Binary Search



Binary Search is an efficient algorithm used to **find the position of a target value** within a **sorted array**. Unlike **linear search**, it repeatedly divides the search interval in half, significantly **reducing the number** of comparisons.

Example 1:

Input: [1, 3, 5, 7, 9]

Output: 7

Approach:

Set **left = 0** **right = nums.length - 1**.

While **left <= right**:

Calculate **middle = Math.floor((left + right) / 2)**.

If **nums[middle] === target**, return **middle**.

If **target < nums[middle]**, discard the right half: **right = middle - 1**.

Else discard the left half: **left = middle + 1**.

If the **target** is **not found** return **-1**.

Time & Space Complexity:

Time Complexity: O(1) (Best Case) when the target is found at the middle initially.

Worst Case: O(log n) the array is halved every iteration.

Space Complexity: O(1) Constant Space No additional data structures.

Dry Run

Input: `nums = [-1, 0, 3, 5, 9, 12], target = 9`

Initial:

`left = 0, right = 5`

Iteration 1:

`middle = Math.floor((0 + 5) / 2) = 2`

`nums[2] = 3`

`target = 9`

`→ target > nums[middle] → left = middle + 1 = 3`

State: `left = 3, right = 5`

Iteration 2:

`middle = Math.floor((3 + 5) / 2) = 4`

`nums[4] = 9`

`target = 9`

`→ target === nums[middle] → return 4`

Loop ends (target found).

Output: `4` (Target found at index 4)

JavaScript

Python

Java

C++

C

C#

```
var search = function(nums, target) {  
  let left = 0;  
  let right = nums.length - 1;  
  
  while (right >= left) {  
    let middle = Math.floor((left + right) / 2);
```

```
    if (target === nums[middle]) {  
        return middle;  
    } else if (target < nums[middle]) {  
        right = middle - 1;  
    } else {  
        left = middle + 1;  
    }  
}  
  
return -1;  
};
```

[Video](#)[Course](#)[Announcements](#)[Discuss doubts](#)[Certificate](#)[Contribute](#)

Binary Search - DSA Notes

Binary Search - DSA Notes

32 of 280 lessons

11% complete

Time/Space Complexity

Arrays - Easy/Medium

Recursion - Easy/Medium

Searching & Sorting - Easy/Medium